

SMART HEALTHCARE PLATFORM

Bridging Healthcare Access Gap Between Tier 2/3 and Metro Cities

Version: 2.0

Date: October 01, 2025

Status: Proposal for Review

Project Type: Healthcare Digital Transformation

Target: Tier 2/3 Cities in India

EXECUTIVE SUMMARY

Smart Healthcare is a comprehensive digital platform designed to democratize healthcare access for patients in Tier 2 and Tier 3 cities across India. The platform connects patients with specialist doctors from metro cities, enabling appointment booking, teleconsultation, digital health records, and seamless payment processing.

Key Impact:

- Bridge the specialist shortage gap (70% deficit in Tier 2/3 cities)
 - Reduce patient travel costs by 80%
 - Enable 24/7 healthcare access
 - Digital health record maintenance
 - Transparent billing and revenue tracking
-

TABLE OF CONTENTS

1. Document Control
2. Problem Statement & Market Opportunity
3. Project Goals & Success Metrics
4. Target Users & Personas
5. Functional Requirements
6. Technical Architecture
7. Database Schema & Models
8. API Specifications
9. Security & Compliance
10. Non-Functional Requirements
11. User Journey Maps

- 12. Technology Stack
- 13. Development Roadmap
- 14. Test Strategy
- 15. Deployment Plan
- 16. Risk Assessment
- 17. Budget & Resources
- 18. Success Metrics & KPIs

1. DOCUMENT CONTROL

Version	Date	Author	Description
1.0	Oct 1, 2025	Prashant Nagar	Initial Draft

Document Purpose:

This document serves as the complete Product Requirements Document (PRD) and Technical Design Document (TDD) for the Smart Healthcare platform, intended for stakeholder review, investor presentation, and development team execution.

2. PROBLEM STATEMENT & MARKET OPPORTUNITY

2.1 Current Healthcare Challenges

For Patients in Tier 2/3 Cities:

- 70% shortage of specialist doctors (Indian Medical Association, 2024)
- Average travel time to metro hospitals: 4-6 hours
- Fragmented medical records across multiple hospitals
- Lack of transparency in billing
- No access to second opinions from top specialists
- Emergency consultation delays

For Doctors:

- Inefficient appointment management leading to revenue loss
- Paper-based record keeping prone to errors
- Difficulty in tracking patient history
- Limited reach beyond local geography

- Administrative overhead in billing and reporting

2.2 Market Opportunity

- **Total Addressable Market (TAM):** 600M+ people in Tier 2/3 cities
- **Serviceable Available Market (SAM):** 150M+ digitally literate healthcare seekers
- **Digital Health Market Growth:** 39% CAGR (2024-2029)
- **Smartphone Penetration:** 65% in target demographicsadd solution
- **Government Push:** National Digital Health Mission (NDHM) integration

3. PROJECT GOALS & SUCCESS METRICS

3.1 Primary Goals

1. **Accessibility:** Enable 100K+ patients to access specialist care within 6 months
2. **Efficiency:** Reduce average appointment waiting time from 7 days to 24 hours
3. **Digital Records:** Maintain lifetime digital health records for all users
4. **Financial Transparency:** 100% trackable payments and automated billing
5. **Teleconsultation:** Enable remote consultations saving patient travel costs

3.2 Success Metrics (6-Month Targets)

Metric	Target	Measurement Method
Active Patients	50,000	User registrations with verified profiles
Onboarded Doctors	500	Verified medical practitioners across 20+ specializations
Monthly Appointments	10,000	Successful bookings completed
Teleconsultations	60% of total	Video/audio consultation completion rate
User Satisfaction	4.2/5.0	Post-consultation ratings
Platform Uptime	99.5%	System availability monitoring
Payment Success Rate	98%	Transaction completion without failures
Avg Response Time	<2 sec	API response time monitoring

4. TARGET USERS & PERSONAS

4.1 Primary Users

Persona 1: Patient - Rajesh Kumar

- **Age:** 42, Small business owner in Bhopal

- **Pain Points:** Father has diabetes, needs regular specialist consultation, nearest endocrinologist 150km away
- **Tech Comfort:** Moderate, uses WhatsApp, Google Pay
- **Goals:** Find trusted specialist, maintain father's medical history, minimize travel

Persona 2: Doctor - Dr. Priya Sharma

- **Age:** 38, Cardiologist in Mumbai
- **Pain Points:** Fully booked locally, wants to serve rural patients, difficulty managing appointments across multiple channels
- **Tech Comfort:** High, uses multiple healthcare apps
- **Goals:** Expand patient base, efficient scheduling, better revenue tracking

Persona 3: Patient - Anita Desai

- **Age:** 28, Teacher in Nashik
- **Pain Points:** Pregnant, needs regular checkups, specialist advice for complications
- **Tech Comfort:** High, comfortable with apps
- **Goals:** Easy appointment booking, access to ob-gyn specialists, track pregnancy reports

4.2 Secondary Users

- **System Administrators:** Platform management and support
- **Hospital Staff:** Coordinate with doctors for in-person follow-ups
- **Healthcare Aggregators:** Potential B2B partnerships

5. FUNCTIONAL REQUIREMENTS

5.1 Patient Features

Core Features

1. User Registration & Authentication

- Email/Phone OTP-based signup
- Profile creation with basic health info
- Password recovery mechanism

2. Doctor Discovery

- Search by specialization, location, language, experience
- Filter by availability, consultation fees, ratings
- View doctor profiles (qualifications, experience, reviews)

3. Appointment Management

- Book appointments by selecting available slots
- Reschedule up to 4 hours before appointment
- Cancel with refund as per cancellation policy
- Automated reminders (24hr, 1hr, 15min before)

4. Medical Records

- Upload prescriptions, lab reports, imaging (PDF, JPEG, PNG)
- View complete medical history chronologically
- Share records with doctors selectively
- Download/print records

5. Teleconsultation

- HD video/audio calls (WebRTC)
- In-call chat for sharing info
- Screen sharing for viewing reports
- Call recording (with consent) for reference

6. Payment & Billing

- Multiple payment options (UPI, Cards, Wallets, NetBanking)
- View billing history and invoices
- Automatic GST invoices generation
- Refund tracking

7. Notifications

- Push notifications (appointment reminders, doctor messages)
- SMS notifications (for low-connectivity users)
- Email notifications (appointment confirmations, reports)

5.2 Doctor Features

Core Features

1. Professional Profile Management

- Complete profile with qualifications, specializations
- Upload medical license and certificates
- Set consultation fees and durations
- Add clinic/hospital affiliations

2. Availability Management

- Set weekly schedule with time slots

- Mark unavailable dates (holidays, conferences)
- Real-time slot blocking on bookings
- Set buffer time between appointments

3. Appointment Dashboard

- View daily/weekly appointment calendar
- Patient list with basic info
- Appointment status tracking (upcoming, completed, cancelled)
- Reschedule/cancel appointments

4. Patient Record Management

- Access patient medical history (with consent)
- View uploaded reports and prescriptions
- Add consultation notes and diagnosis
- Track patient progress over visits

5. Prescription Management

- Digital prescription creation
- Medicine database integration
- Lab test recommendations
- E-prescription PDF generation with digital signature

6. Teleconsultation Tools

- Initiate video/audio calls
- Access patient records during consultation
- Real-time prescription writing
- Post-consultation summary notes

7. Financial Dashboard

- Daily/monthly revenue reports
- Appointment-wise earnings breakdown
- TDS and GST reports
- Payout management and bank transfer tracking

8. Analytics & Insights

- Patient demographics
- Consultation patterns
- Peak hours and scheduling efficiency
- Patient satisfaction ratings

5.3 Admin Features

Core Features

1. User Management

- View/manage all patients and doctors
- Verify doctor credentials and licenses
- Suspend/ban accounts for policy violations
- Reset passwords and handle support escalations

2. Platform Monitoring

- Real-time system health dashboard
- API performance metrics
- Error logs and debugging tools
- Database query performance

3. Content Management

- Manage specialization categories
- Update app banners and announcements
- Maintain FAQ and help content
- Push notifications to user segments

4. Financial Oversight

- View platform-wide revenue
- Doctor payout approvals
- Refund processing
- Commission and fee structure management

5. Reporting & Analytics

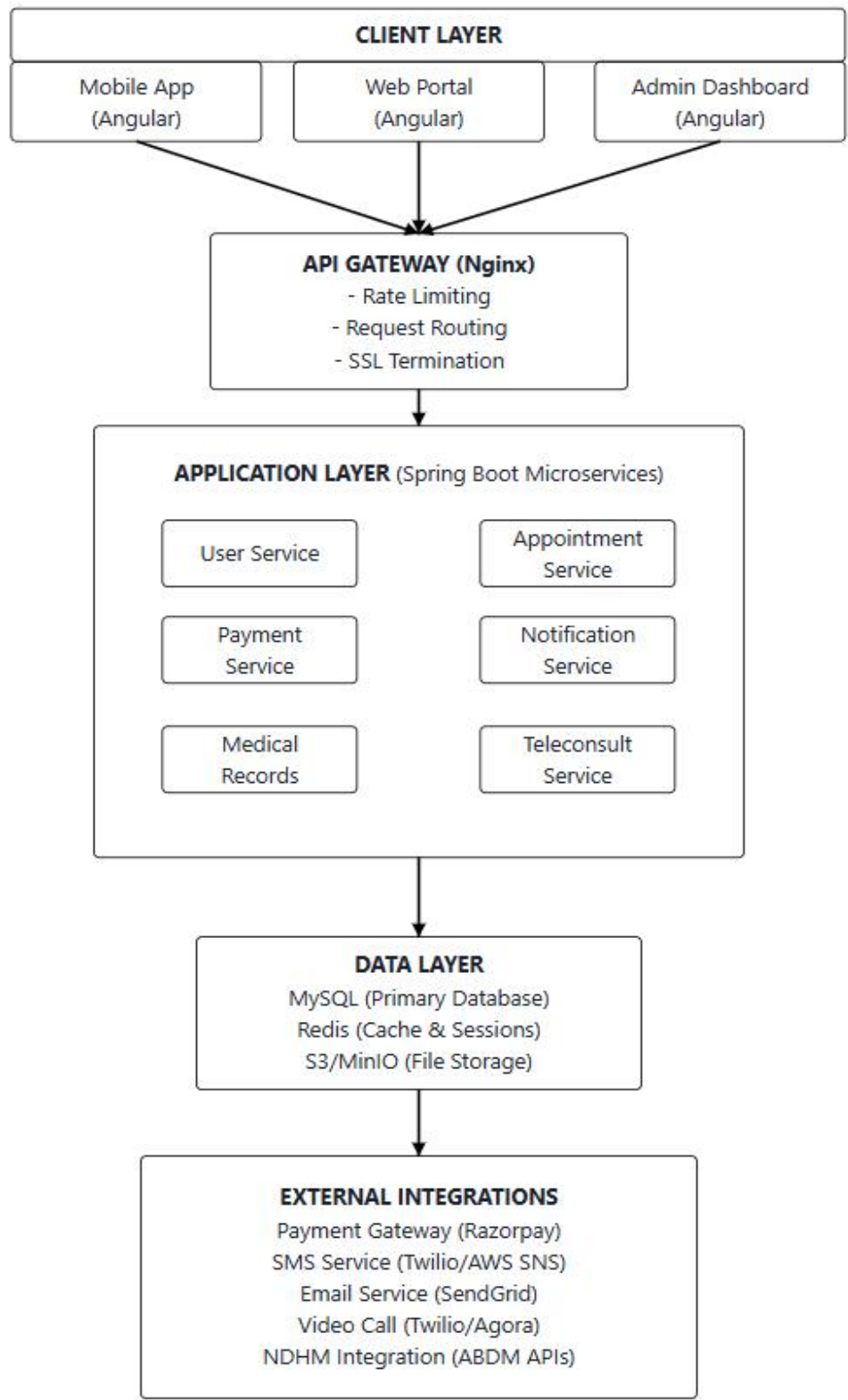
- User growth reports (DAU, MAU, registrations)
- Appointment trends and conversion rates
- Geographic distribution of users
- Revenue and transaction reports
- Doctor performance metrics

6. Audit & Compliance

- User activity logs
- Data access logs (DPDP compliance)
- System change logs
- Security incident tracking

6. TECHNICAL ARCHITECTURE

6.1 High-Level Architecture



6.2 Microservices Architecture

Service Breakdown

1. User Service

- User registration and authentication (JWT)
- Profile management (Patient, Doctor, Admin)
- Role-based access control

2. Appointment Service

- Appointment booking and scheduling
- Slot management and availability
- Cancellation and rescheduling

3. Payment Service

- Payment gateway integration
- Transaction processing and refunds
- Invoice generation
- Payout management

4. Notification Service

- Push notifications (FCM)
- SMS notifications
- Email notifications
- Notification templates and scheduling

5. Medical Records Service

- Record storage and retrieval
- File upload management
- Prescription management

- Data encryption and security

6. Teleconsultation Service

- Video/audio call integration
- Call session management
- Call recording and storage
- Consultation notes

7. Analytics Service

- Data aggregation and reporting
- Dashboard metrics
- Business intelligence

7. DATABASE SCHEMA & MODELS

7.1 Core Tables

User Table

```
sql

CREATE TABLE user (
  user_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_name VARCHAR(100) NOT NULL,
  user_email VARCHAR(255) UNIQUE NOT NULL,
  user_password VARCHAR(255) NOT NULL,
  phone_number VARCHAR(15) UNIQUE,
  role ENUM('ADMIN', 'DOCTOR', 'PATIENT') NOT NULL,
  is_verified BOOLEAN DEFAULT FALSE,
  is_active BOOLEAN DEFAULT TRUE,
  last_login TIMESTAMP NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_email (user_email),
  INDEX idx_phone (phone_number),
  INDEX idx_role (role)
);
```

Patient Table

```
sql
```

```
CREATE TABLE patient (  
  patient_id BIGINT PRIMARY KEY,  
  user_id BIGINT UNIQUE NOT NULL,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50),  
  date_of_birth DATE,  
  gender ENUM('MALE', 'FEMALE', 'OTHER'),  
  phone_number VARCHAR(15),  
  address TEXT,  
  city VARCHAR(100),  
  state VARCHAR(100),  
  pincode VARCHAR(10),  
  emergency_contact_name VARCHAR(100),  
  emergency_phone_number VARCHAR(15),  
  blood_group VARCHAR(5),  
  allergies TEXT,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE CASCADE,  
  INDEX idx_city (city),  
  INDEX idx_phone (phone_number)  
);
```

Doctor Table

sql

```
CREATE TABLE doctor (  
  doctor_id BIGINT PRIMARY KEY,  
  user_id BIGINT UNIQUE NOT NULL,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50),  
  specialization VARCHAR(100) NOT NULL,  
  license_number VARCHAR(50) UNIQUE NOT NULL,  
  qualification TEXT,  
  experience_years INT,  
  phone VARCHAR(15),  
  consultation_fee DECIMAL(10,2),  
  consultation_duration INT DEFAULT 30, -- minutes  
  bio TEXT,  
  hospital_affiliation VARCHAR(255),  
  address TEXT,  
  city VARCHAR(100),  
  state VARCHAR(100),  
  languages_spoken VARCHAR(255),  
  rating DECIMAL(3,2) DEFAULT 0.0,  
  total_consultations INT DEFAULT 0,  
  is_verified BOOLEAN DEFAULT FALSE,  
  is_available BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE CASCADE,  
  INDEX idx_specialization (specialization),  
  INDEX idx_city (city),  
  INDEX idx_rating (rating)  
);
```

Admin Table

sql

```
CREATE TABLE admin (
  admin_id BIGINT PRIMARY KEY,
  user_id BIGINT UNIQUE NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50),
  department VARCHAR(100),
  access_level INT DEFAULT 1,
  phone VARCHAR(15),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE CASCADE
);
```

Appointment Table (NEW)

```
sql

CREATE TABLE appointment (
  appointment_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  patient_id BIGINT NOT NULL,
  doctor_id BIGINT NOT NULL,
  appointment_date DATE NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  appointment_type ENUM('IN_PERSON', 'TELECONSULTATION') DEFAULT 'TELECONSULTATION',
  status ENUM('SCHEDULED', 'CONFIRMED', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED', 'NO_SHOW') DEFAULT 'SCHEDULED',
  consultation_fee DECIMAL(10,2) NOT NULL,
  payment_status ENUM('PENDING', 'PAID', 'REFUNDED', 'FAILED') DEFAULT 'PENDING',
  symptoms TEXT,
  notes TEXT,
  cancellation_reason TEXT,
  cancelled_by BIGINT NULL,
  cancelled_at TIMESTAMP NULL,
  reminder_sent BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (patient_id) REFERENCES patient(patient_id) ON DELETE CASCADE,
  FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id) ON DELETE CASCADE,
  INDEX idx_patient (patient_id),
  INDEX idx_doctor (doctor_id),
  INDEX idx_date (appointment_date),
  INDEX idx_status (status),
  UNIQUE KEY unique_doctor_slot (doctor_id, appointment_date, start_time)
);
```

Medical Record Table (NEW)

```
sql

CREATE TABLE medical_record (
  record_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  patient_id BIGINT NOT NULL,
  doctor_id BIGINT NULL,
  appointment_id BIGINT NULL,
  record_type ENUM('PRESCRIPTION', 'LAB_REPORT', 'IMAGING', 'DIAGNOSIS', 'VACCINATION', 'OTHER') NOT
  record_date DATE NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  diagnosis TEXT,
  file_url VARCHAR(500),
  file_name VARCHAR(255),
  file_type VARCHAR(50),
  file_size INT,
  is_shared BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (patient_id) REFERENCES patient(patient_id) ON DELETE CASCADE,
  FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id) ON DELETE SET NULL,
  FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id) ON DELETE SET NULL,
  INDEX idx_patient (patient_id),
  INDEX idx_date (record_date),
  INDEX idx_type (record_type)
);
```

Prescription Table (NEW)

```
sql
```

```
CREATE TABLE prescription (
  prescription_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  appointment_id BIGINT NOT NULL,
  patient_id BIGINT NOT NULL,
  doctor_id BIGINT NOT NULL,
  prescription_date DATE NOT NULL,
  diagnosis TEXT,
  notes TEXT,
  follow_up_date DATE NULL,
  digital_signature VARCHAR(500),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE,
  FOREIGN KEY (patient_id) REFERENCES patient(patient_id) ON DELETE CASCADE,
  FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id) ON DELETE CASCADE,
  INDEX idx_patient (patient_id),
  INDEX idx_appointment (appointment_id)
);
```

Prescription Medicine Table (NEW)

```
sql

CREATE TABLE prescription_medicine (
  medicine_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  prescription_id BIGINT NOT NULL,
  medicine_name VARCHAR(255) NOT NULL,
  dosage VARCHAR(100) NOT NULL,
  frequency VARCHAR(100) NOT NULL, -- e.g., "Twice a day"
  duration VARCHAR(50) NOT NULL, -- e.g., "7 days"
  instructions TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (prescription_id) REFERENCES prescription(prescription_id) ON DELETE CASCADE,
  INDEX idx_prescription (prescription_id)
);
```

Lab Test Table (NEW)

```
sql
```

```
CREATE TABLE lab_test (
  test_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  prescription_id BIGINT NOT NULL,
  test_name VARCHAR(255) NOT NULL,
  test_category VARCHAR(100),
  instructions TEXT,
  is_completed BOOLEAN DEFAULT FALSE,
  result_file_url VARCHAR(500),
  completed_date DATE NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (prescription_id) REFERENCES prescription(prescription_id) ON DELETE CASCADE,
  INDEX idx_prescription (prescription_id)
);
```

Payment Transaction Table (NEW)

```
sql

CREATE TABLE payment_transaction (
  transaction_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  appointment_id BIGINT NOT NULL,
  patient_id BIGINT NOT NULL,
  doctor_id BIGINT NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  platform_fee DECIMAL(10,2) DEFAULT 0.00,
  doctor_payout DECIMAL(10,2) NOT NULL,
  payment_method ENUM('UPI', 'CARD', 'NET_BANKING', 'WALLET') NOT NULL,
  transaction_status ENUM('INITIATED', 'SUCCESS', 'FAILED', 'REFUNDED') DEFAULT 'INITIATED',
  payment_gateway_id VARCHAR(255),
  payment_gateway_response TEXT,
  refund_amount DECIMAL(10,2) DEFAULT 0.00,
  refund_date TIMESTAMP NULL,
  invoice_url VARCHAR(500),
  gst_amount DECIMAL(10,2) DEFAULT 0.00,
  transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE,
  FOREIGN KEY (patient_id) REFERENCES patient(patient_id) ON DELETE CASCADE,
  FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id) ON DELETE CASCADE,
  INDEX idx_appointment (appointment_id),
  INDEX idx_status (transaction_status),
  INDEX idx_date (transaction_date)
);
```

Doctor Availability Table (NEW)

sql

```
CREATE TABLE doctor_availability (  
  availability_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  doctor_id BIGINT NOT NULL,  
  day_of_week ENUM('MONDAY', 'TUESDAY', 'WEDNESDAY', 'THURSDAY', 'FRIDAY', 'SATURDAY', 'SUNDAY') NOT NULL,  
  start_time TIME NOT NULL,  
  end_time TIME NOT NULL,  
  slot_duration INT DEFAULT 30, -- minutes  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id) ON DELETE CASCADE,  
  INDEX idx_doctor (doctor_id),  
  INDEX idx_day (day_of_week)  
);
```

Notification Table (NEW)

sql

```
CREATE TABLE notification (  
  notification_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  user_id BIGINT NOT NULL,  
  notification_type ENUM('APPOINTMENT_REMINDER', 'APPOINTMENT_CONFIRMED', 'APPOINTMENT_CANCELLATION') NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  message TEXT NOT NULL,  
  is_read BOOLEAN DEFAULT FALSE,  
  notification_channel ENUM('PUSH', 'SMS', 'EMAIL') NOT NULL,  
  reference_id BIGINT NULL, -- appointment_id or other reference  
  sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  read_at TIMESTAMP NULL,  
  FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE CASCADE,  
  INDEX idx_user (user_id),  
  INDEX idx_read (is_read),  
  INDEX idx_type (notification_type)  
);
```

Review & Rating Table (NEW)

sql

```
CREATE TABLE review (  
  review_id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  appointment_id BIGINT NOT NULL,  
  patient_id BIGINT NOT NULL,  
  doctor_id BIGINT NOT NULL,  
  rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5),  
  review_text TEXT,  
  is_anonymous BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id) ON DELETE CASCADE,  
  FOREIGN KEY (patient_id) REFERENCES patient(patient_id) ON DELETE CASCADE,  
  FOREIGN KEY (doctor_id) REFERENCES doctor(doctor_id) ON DELETE CASCADE,  
  INDEX idx_doctor (doctor_id),  
  INDEX idx_rating (rating),  
  UNIQUE KEY unique_appointment_review (appointment_id)  
);
```

7.2 Database Relationships Summary

User (1) → (1) Patient
User (1) → (1) Doctor
User (1) → (1) Admin
Patient (1) → (Many) Appointment
Doctor (1) → (Many) Appointment
Appointment (1) → (1) Payment Transaction
Appointment (1) → (0..1) Prescription
Prescription (1) → (Many) Prescription Medicine
Prescription (1) → (Many) Lab Test
Patient (1) → (Many) Medical Record
Doctor (1) → (Many) Doctor Availability
Patient (1) → (Many) Review
Doctor (1) → (Many) Review

8. API SPECIFICATIONS

8.1 Authentication APIs

POST /api/v1/auth/register

Description: Register new user (Patient/Doctor)

Request Body:

json

```
{
  "userName": "Rajesh Kumar",
  "email": "rajesh.kumar@email.com",
  "password": "SecurePass@123",
  "phoneNumber": "+919876543210",
  "role": "PATIENT"
}
```

Response (201 Created):

```
json

{
  "success": true,
  "message": "User registered successfully. Please verify your email.",
  "data": {
    "userId": 12345,
    "email": "rajesh.kumar@email.com",
    "role": "PATIENT",
    "isVerified": false
  }
}
```

POST /api/v1/auth/login

Description: User login with JWT token generation

Request Body:

```
json

{
  "email": "rajesh.kumar@email.com",
  "password": "SecurePass@123"
}
```

Response (200 OK):

```
json
```

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "userId": 12345,
    "userName": "Rajesh Kumar",
    "email": "rajesh.kumar@email.com",
    "role": "PATIENT",
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "expiresIn": 3600
  }
}
```

POST /api/v1/auth/logout

Description: Logout user and invalidate token

Headers: `Authorization: Bearer <token>`

Response (200 OK):

```
json

{
  "success": true,
  "message": "Logout successful"
}
```

8.2 Doctor APIs

GET /api/v1/doctors

Description: Get list of doctors with filters

Query Parameters:

- `specialization` (optional): Filter by specialization
- `city` (optional): Filter by city
- `minRating` (optional): Minimum rating
- `page` (default: 0)
- `size` (default: 20)

Response (200 OK):

```
json
```

```
{
  "success": true,
  "data": {
    "doctors": [
      {
        "doctorId": 101,
        "firstName": "Priya",
        "lastName": "Sharma",
        "specialization": "Cardiology",
        "qualification": "MD, DM Cardiology",
        "experienceYears": 12,
        "consultationFee": 800.00,
        "rating": 4.7,
        "totalConsultations": 1250,
        "city": "Mumbai",
        "languages": "Hindi, English, Marathi",
        "isAvailable": true
      }
    ],
    "pagination": {
      "currentPage": 0,
      "totalPages": 5,
      "totalElements": 95,
      "pageSize": 20
    }
  }
}
```

GET /api/v1/doctors/{id}

Description: Get doctor details by ID

Response (200 OK):

json

```
{
  "success": true,
  "data": {
    "doctorId": 101,
    "firstName": "Priya",
    "lastName": "Sharma",
    "specialization": "Cardiology",
    "qualification": "MD, DM Cardiology, FACC",
    "experienceYears": 12,
    "consultationFee": 800.00,
    "consultationDuration": 30,
    "bio": "Experienced cardiologist specializing in interventional cardiology and preventive cardiac care",
    "hospitalAffiliation": "Lilavati Hospital, Mumbai",
    "rating": 4.7,
    "totalConsultations": 1250,
    "city": "Mumbai",
    "languages": "Hindi, English, Marathi",
    "isAvailable": true,
    "availability": [
      {
        "dayOfWeek": "MONDAY",
        "startTime": "09:00",
        "endTime": "17:00"
      },
      {
        "dayOfWeek": "WEDNESDAY",
        "startTime": "09:00",
        "endTime": "17:00"
      }
    ]
  }
}
```

PUT /api/v1/doctors/{id}

Description: Update doctor profile (Doctor only)

Headers: Authorization: Bearer <token>

Request Body:

json

```
{  
  "consultationFee": 900.00,  
  "bio": "Updated bio",  
  "isAvailable": true  
}
```

Response (200 OK):

```
json  
  
{  
  "success": true,  
  "message": "Doctor profile updated successfully",  
  "data": {  
    "doctorId": 101,  
    "consultationFee": 900.00  
  }  
}
```

8.3 Patient APIs

GET /api/v1/patients/{id}

Description: Get patient details

Headers: Authorization: Bearer <token>

Response (200 OK):

```
json
```

```
{
  "success": true,
  "data": {
    "patientId": 5001,
    "firstName": "Rajesh",
    "lastName": "Kumar",
    "dateOfBirth": "1982-05-15",
    "gender": "MALE",
    "phoneNumber": "+919876543210",
    "address": "123 Main Street, Bhopal",
    "city": "Bhopal",
    "state": "Madhya Pradesh",
    "bloodGroup": "O+",
    "allergies": "Penicillin",
    "emergencyContactName": "Sunita Kumar",
    "emergencyPhoneNumber": "+919876543211"
  }
}
```

PUT /api/v1/patients/{id}

Description: Update patient profile

Headers: Authorization: Bearer <token>

Request Body:

```
json

{
  "phoneNumber": "+919876543210",
  "address": "Updated address",
  "bloodGroup": "O+",
  "allergies": "Penicillin, Sulfa drugs"
}
```

Response (200 OK):

```
json

{
  "success": true,
  "message": "Patient profile updated successfully"
}
```

8.4 Appointment APIs

POST /api/v1/appointments

Description: Book new appointment

Headers: Authorization: Bearer <token>

Request Body:

```
json

{
  "doctorId": 101,
  "appointmentDate": "2025-10-15",
  "startTime": "10:00",
  "appointmentType": "TELECONSULTATION",
  "symptoms": "Chest pain and breathing difficulty"
}
```

Response (201 Created):

```
json

{
  "success": true,
  "message": "Appointment booked successfully",
  "data": {
    "appointmentId": 8001,
    "patientId": 5001,
    "doctorId": 101,
    "doctorName": "Dr. Priya Sharma",
    "appointmentDate": "2025-10-15",
    "startTime": "10:00",
    "endTime": "10:30",
    "appointmentType": "TELECONSULTATION",
    "status": "SCHEDULED",
    "consultationFee": 800.00,
    "paymentStatus": "PENDING"
  }
}
```

GET /api/v1/appointments/{id}

Description: Get appointment details

Headers: Authorization: Bearer <token>

Response (200 OK):

json

```
{
  "success": true,
  "data": {
    "appointmentId": 8001,
    "patient": {
      "patientId": 5001,
      "name": "Rajesh Kumar",
      "age": 42,
      "gender": "MALE"
    },
    "doctor": {
      "doctorId": 101,
      "name": "Dr. Priya Sharma",
      "specialization": "Cardiology"
    },
    "appointmentDate": "2025-10-15",
    "startTime": "10:00",
    "endTime": "10:30",
    "appointmentType": "TELECONSULTATION",
    "status": "CONFIRMED",
    "consultationFee": 800.00,
    "paymentStatus": "PAID",
    "symptoms": "Chest pain and breathing difficulty"
  }
}
```

GET /api/v1/appointments/patient/{patientId}

Description: Get all appointments for a patient

Headers: Authorization: Bearer <token>

Query Parameters:

- status (optional): Filter by status
- fromDate (optional): Start date
- toDate (optional): End date

Response (200 OK):

json

```
{
  "success": true,
  "data": {
    "appointments": [
      {
        "appointmentId": 8001,
        "doctorName": "Dr. Priya Sharma",
        "specialization": "Cardiology",
        "appointmentDate": "2025-10-15",
        "startTime": "10:00",
        "status": "SCHEDULED",
        "appointmentType": "TELECONSULTATION"
      }
    ],
    "totalAppointments": 15,
    "upcomingCount": 2,
    "completedCount": 12,
    "cancelledCount": 1
  }
}
```

GET /api/v1/appointments/doctor/{doctorId}

Description: Get all appointments for a doctor

Headers: Authorization: Bearer <token>

Response (200 OK):

json

```
{
  "success": true,
  "data": {
    "appointments": [
      {
        "appointmentId": 8001,
        "patientName": "Rajesh Kumar",
        "patientAge": 42,
        "appointmentDate": "2025-10-15",
        "startTime": "10:00",
        "status": "SCHEDULED",
        "symptoms": "Chest pain and breathing difficulty"
      }
    ],
    "todayCount": 8,
    "weekCount": 32,
    "monthCount": 125
  }
}
```

PUT /api/v1/appointments/{id}/reschedule

Description: Reschedule appointment

Headers: (Authorization: Bearer <token>)

Request Body:

```
json

{
  "newAppointmentDate": "2025-10-16",
  "newStartTime": "14:00"
}
```

Response (200 OK):

```
json
```

```
{
  "success": true,
  "message": "Appointment rescheduled successfully",
  "data": {
    "appointmentId": 8001,
    "appointmentDate": "2025-10-16",
    "startTime": "14:00"
  }
}
```

DELETE /api/v1/appointments/{id}

Description: Cancel appointment

Headers: Authorization: Bearer <token>

Request Body:

```
json

{
  "cancellationReason": "Personal emergency"
}
```

Response (200 OK):

```
json

{
  "success": true,
  "message": "Appointment cancelled successfully",
  "data": {
    "appointmentId": 8001,
    "status": "CANCELLED",
    "refundAmount": 800.00,
    "refundStatus": "INITIATED"
  }
}
```

GET /api/v1/appointments/slots/doctor/{doctorId}

Description: Get available slots for a doctor

Query Parameters:

- ♦ date (required): Date in YYYY-MM-DD format

Response (200 OK):

```
json
{
  "success": true,
  "data": {
    "doctorId": 101,
    "date": "2025-10-15",
    "availableSlots": [
      {
        "startTime": "09:00",
        "endTime": "09:30",
        "isAvailable": true
      },
      {
        "startTime": "09:30",
        "endTime": "10:00",
        "isAvailable": true
      },
      {
        "startTime": "10:00",
        "endTime": "10:30",
        "isAvailable": false
      }
    ]
  }
}
```

8.5 Medical Records APIs

POST /api/v1/records

Description: Add medical record

Headers:

- Authorization: Bearer <token>
- Content-Type: multipart/form-data

Form Data:

- patientId: 5001
- recordType: PRESCRIPTION
- recordDate: 2025-10-15
- title: Blood Test Report

- ♦ `description`: Complete Blood Count
- ♦ `file`: (file upload)

Response (201 Created):

```
json
{
  "success": true,
  "message": "Medical record added successfully",
  "data": {
    "recordId": 3001,
    "patientId": 5001,
    "recordType": "LAB_REPORT",
    "title": "Blood Test Report",
    "fileUrl": "https://storage.example.com/records/3001_blood_test.pdf",
    "recordDate": "2025-10-15"
  }
}
```

GET /api/v1/records/patient/{patientId}

Description: Get all medical records for a patient

Headers: `Authorization: Bearer <token>`

Query Parameters:

- ♦ `recordType` (optional): Filter by type
- ♦ `fromDate` (optional)
- ♦ `toDate` (optional)

Response (200 OK):

```
json
```

```
{
  "success": true,
  "data": {
    "records": [
      {
        "recordId": 3001,
        "recordType": "LAB_REPORT",
        "title": "Blood Test Report",
        "recordDate": "2025-10-15",
        "doctorName": "Dr. Priya Sharma",
        "fileUrl": "https://storage.example.com/records/3001_blood_test.pdf",
        "fileName": "blood_test_report.pdf"
      },
      {
        "recordId": 3002,
        "recordType": "PRESCRIPTION",
        "title": "Cardiac Medication",
        "recordDate": "2025-10-15",
        "doctorName": "Dr. Priya Sharma",
        "fileUrl": "https://storage.example.com/records/3002_prescription.pdf"
      }
    ],
    "totalRecords": 25
  }
}
```

GET /api/v1/records/{id}

Description: Get specific medical record

Headers: Authorization: Bearer <token>

Response (200 OK):

json

```
{
  "success": true,
  "data": {
    "recordId": 3001,
    "patientId": 5001,
    "patientName": "Rajesh Kumar",
    "doctorId": 101,
    "doctorName": "Dr. Priya Sharma",
    "recordType": "LAB_REPORT",
    "recordDate": "2025-10-15",
    "title": "Blood Test Report",
    "description": "Complete Blood Count",
    "fileUrl": "https://storage.example.com/records/3001_blood_test.pdf",
    "fileName": "blood_test_report.pdf",
    "fileSize": 2048576
  }
}
```

DELETE /api/v1/records/{id}

Description: Delete medical record

Headers: Authorization: Bearer <token>

Response (200 OK):

```
json

{
  "success": true,
  "message": "Medical record deleted successfully"
}
```

8.6 Prescription APIs

POST /api/v1/prescriptions

Description: Create prescription (Doctor only)

Headers: Authorization: Bearer <token>

Request Body:

```
json
```

```
{
  "appointmentId": 8001,
  "patientId": 5001,
  "diagnosis": "Hypertension Stage 1",
  "notes": "Patient advised lifestyle modifications",
  "followUpDate": "2025-11-15",
  "medicines": [
    {
      "medicineName": "Amlodipine 5mg",
      "dosage": "1 tablet",
      "frequency": "Once daily",
      "duration": "30 days",
      "instructions": "Take in the morning after breakfast"
    },
    {
      "medicineName": "Aspirin 75mg",
      "dosage": "1 tablet",
      "frequency": "Once daily",
      "duration": "30 days",
      "instructions": "Take after dinner"
    }
  ],
  "labTests": [
    {
      "testName": "Lipid Profile",
      "testCategory": "Blood Test",
      "instructions": "Fasting required for 12 hours"
    }
  ]
}
```

Response (201 Created):

```
json
{
  "success": true,
  "message": "Prescription created successfully",
  "data": {
    "prescriptionId": 9001,
    "appointmentId": 8001,
    "prescriptionDate": "2025-10-15",
    "pdfUrl": "https://storage.example.com/prescriptions/9001.pdf"
  }
}
```

GET /api/v1/prescriptions/appointment/{appointmentId}

Description: Get prescription for an appointment

Headers:

Authorization: Bearer <token>

Response (200 OK):

json

```
{
  "success": true,
  "data": {
    "prescriptionId": 9001,
    "appointmentId": 8001,
    "patient": {
      "name": "Rajesh Kumar",
      "age": 42,
      "gender": "MALE"
    },
    "doctor": {
      "name": "Dr. Priya Sharma",
      "specialization": "Cardiology",
      "licenseNumber": "MH12345"
    },
    "prescriptionDate": "2025-10-15",
    "diagnosis": "Hypertension Stage 1",
    "notes": "Patient advised lifestyle modifications",
    "followUpDate": "2025-11-15",
    "medicines": [
      {
        "medicineName": "Amlodipine 5mg",
        "dosage": "1 tablet",
        "frequency": "Once daily",
        "duration": "30 days",
        "instructions": "Take in the morning after breakfast"
      }
    ],
    "labTests": [
      {
        "testName": "Lipid Profile",
        "testCategory": "Blood Test",
        "isCompleted": false
      }
    ],
    "pdfUrl": "https://storage.example.com/prescriptions/9001.pdf"
  }
}
```

8.7 Payment APIs

POST /api/v1/payments/initiate Description:

Initiate payment for appointment **Headers:**

Authorization: Bearer <token>

Request Body:

```
json

{
  "appointmentId": 8001,
  "paymentMethod": "UPI"
}
```

Response (200 OK):

```
json

{
  "success": true,
  "message": "Payment initiated",
  "data": {
    "transactionId": 7001,
    "appointmentId": 8001,
    "amount": 800.00,
    "gstAmount": 144.00,
    "totalAmount": 944.00,
    "paymentGatewayOrderId": "order_MxYz123456",
    "paymentUrl": "https://razorpay.com/checkout/order_MxYz123456"
  }
}
```

POST /api/v1/payments/callback

Description: Payment gateway callback (webhook)

Request Body:

```
json

{
  "transactionId": 7001,
  "paymentGatewayId": "pay_ABC123456",
  "status": "SUCCESS",
  "paymentMethod": "UPI"
}
```

Response (200 OK):

```
json
```

```
{
  "success": true,
  "message": "Payment processed successfully",
  "data": {
    "transactionId": 7001,
    "transactionStatus": "SUCCESS",
    "invoiceUrl": "https://storage.example.com/invoices/7001.pdf"
  }
}
```

GET /api/v1/payments/patient/{patientId}

Description: Get payment history for patient

Headers: Authorization: Bearer <token>

Response (200 OK):

```
json
{
  "success": true,
  "data": {
    "transactions": [
      {
        "transactionId": 7001,
        "appointmentId": 8001,
        "doctorName": "Dr. Priya Sharma",
        "amount": 944.00,
        "transactionStatus": "SUCCESS",
        "paymentMethod": "UPI",
        "transactionDate": "2025-10-15T09:30:00",
        "invoiceUrl": "https://storage.example.com/invoices/7001.pdf"
      }
    ],
    "totalSpent": 12500.00,
    "totalTransactions": 15
  }
}
```

POST /api/v1/payments/{transactionId}/refund

Description: Process refund (Admin only)

Headers: Authorization: Bearer <token>

Request Body:

```
json

{
  "refundAmount": 800.00,
  "reason": "Appointment cancelled by doctor"
}
```

Response (200 OK):

```
json

{
  "success": true,
  "message": "Refund initiated successfully",
  "data": {
    "transactionId": 7001,
    "refundAmount": 800.00,
    "refundStatus": "INITIATED"
  }
}
```

8.8 Notification APIs

POST /api/v1/notifications/send

Description: Send notification (System/Admin only)

Request Body:

```
json

{
  "userId": 5001,
  "notificationType": "APPOINTMENT_REMINDER",
  "title": "Appointment Reminder",
  "message": "Your appointment with Dr. Priya Sharma is scheduled for tomorrow at 10:00 AM",
  "notificationChannel": "PUSH",
  "referenceId": 8001
}
```

Response (201 Created):

```
json
```

```
{
  "success": true,
  "message": "Notification sent successfully",
  "data": {
    "notificationId": 6001
  }
}
```

GET /api/v1/notifications/user/{userId}

Description: Get notifications for user

Headers: (Authorization: Bearer <token>)

Query Parameters:

- (isRead) (optional): Filter by read status
- (limit) (default: 50)

Response (200 OK):

```
json

{
  "success": true,
  "data": {
    "notifications": [
      {
        "notificationId": 6001,
        "notificationType": "APPOINTMENT_REMINDER",
        "title": "Appointment Reminder",
        "message": "Your appointment with Dr. Priya Sharma is tomorrow",
        "isRead": false,
        "sentAt": "2025-10-14T18:00:00"
      }
    ],
    "unreadCount": 5,
    "totalCount": 42
  }
}
```

PUT /api/v1/notifications/{id}/read

Description: Mark notification as read

Headers: (Authorization: Bearer <token>)

Response (200 OK):

```
json

{
  "success": true,
  "message": "Notification marked as read"
}
```

8.9 Review APIs

POST /api/v1/reviews

Description: Add review for appointment

Headers: Authorization: Bearer <token>

Request Body:

```
json

{
  "appointmentId": 8001,
  "rating": 5,
  "reviewText": "Excellent consultation. Doctor was very patient and thorough.",
  "isAnonymous": false
}
```

Response (201 Created):

```
json

{
  "success": true,
  "message": "Review submitted successfully",
  "data": {
    "reviewId": 4001,
    "appointmentId": 8001,
    "rating": 5
  }
}
```

GET /api/v1/reviews/doctor/{doctorId}

Description: Get reviews for a doctor

Query Parameters:

- `page` (default: 0)
- `size` (default: 20)

Response (200 OK):

```
json
{
  "success": true,
  "data": {
    "reviews": [
      {
        "reviewId": 4001,
        "patientName": "Rajesh Kumar",
        "rating": 5,
        "reviewText": "Excellent consultation",
        "createdAt": "2025-10-15T12:00:00"
      }
    ],
    "averageRating": 4.7,
    "totalReviews": 1250,
    "ratingDistribution": {
      "5": 850,
      "4": 300,
      "3": 70,
      "2": 20,
      "1": 10
    }
  }
}
```

9. SECURITY & COMPLIANCE

9.1 Authentication & Authorization

JWT Implementation:

- Access Token validity: 1 hour
- Refresh Token validity: 7 days
- Token stored securely (HttpOnly cookies for web)
- Role-based access control (RBAC) for all APIs

Password Security:

- BCrypt hashing with salt (cost factor: 12)

- Minimum 8 characters with complexity requirements
- Password reset via OTP (valid for 10 minutes)

9.2 Data Protection

Encryption:

- TLS 1.3 for all API communications
- AES-256 encryption for sensitive data at rest
- Database encryption for medical records
- Secure file storage with access controls

Data Privacy (DPDP Compliance):

- Explicit consent for data collection
- Right to access personal data
- Right to delete account and data (within 30 days)
- Data anonymization for analytics
- Audit logs for all data access

9.3 API Security

- Rate limiting: 100 requests/minute per user
- API key authentication for service-to-service calls
- Input validation and sanitization
- SQL injection prevention (Prepared statements)
- XSS protection (Content Security Policy)
- CORS configuration for allowed origins

9.4 Medical Data Security

- HIPAA-style guidelines (though not mandatory in India)
- Access logs for all medical record views
- Doctor verification before accessing patient records
- Patient consent required for data sharing
- Secure file upload with virus scanning
- Automatic session timeout after 15 minutes of inactivity

9.5 Payment Security

- PCI DSS compliant payment gateway

- No card details stored on servers
 - Tokenization for recurring payments
 - 3D Secure authentication for cards
 - Fraud detection and prevention
-

10. NON-FUNCTIONAL REQUIREMENTS

10.1 Performance

- API response time: < 2 seconds (95th percentile)
- Page load time: < 3 seconds
- Video call latency: < 300ms
- Database query optimization with indexing
- CDN for static assets

10.2 Scalability

- Horizontal scaling for microservices
- Load balancer (Nginx/AWS ALB)
- Database read replicas
- Redis caching for frequently accessed data
- Support for 100K concurrent users

10.3 Availability

- 99.9% uptime SLA (8.76 hours downtime/year)
- Multi-AZ deployment for redundancy
- Automated failover mechanisms
- Regular database backups (every 6 hours)
- Disaster recovery plan with RTO: 4 hours, RPO: 1 hour

10.4 Usability

- Mobile-first responsive design
- Support for Hindi and English languages
- Accessibility compliance (WCAG 2.1 Level AA)
- Low bandwidth mode for 2G/3G networks
- Offline functionality for viewing records

10.5 Monitoring

- Application performance monitoring (APM)
 - Real-time error tracking
 - User analytics and behavior tracking
 - Server health monitoring
 - Automated alerts for critical issues
-

11. USER JOURNEY MAPS

11.1 Patient Journey - First Consultation

1. Discovery Phase

- Download app / Visit website
- Sign up with phone/email
- Complete profile (age, location, medical history)

2. Search & Selection

- Search doctors by symptoms/specialization
- Filter by location, fees, ratings
- View doctor profiles and reviews
- Check available slots

3. Booking

- Select date and time slot
- Choose consultation type (teleconsultation/in-person)
- Enter symptoms and reason for visit
- Proceed to payment

4. Payment

- Choose payment method
- Complete payment securely
- Receive booking confirmation via SMS/email/push

5. Pre-Appointment

- Upload relevant medical reports (optional)
- Receive appointment reminders
- Access teleconsultation link (if applicable)

6. Consultation

- Join video call at scheduled time
- Discuss symptoms with doctor
- Doctor reviews uploaded reports
- Receive diagnosis and treatment plan

7. Post-Consultation

- Receive digital prescription
- View recommended lab tests
- Download prescription PDF
- Book follow-up appointment (if needed)
- Rate and review doctor

8. Follow-Up

- Upload lab reports when completed
- Access complete consultation history
- Manage all medical records in one place

11.2 Doctor Journey - Daily Workflow

1. Morning Setup

- Login to dashboard
- Review today's appointments
- Check pending patient queries
- Update availability if needed

2. Pre-Consultation Preparation

- Review patient profile and medical history
- Check uploaded reports and past prescriptions
- Prepare consultation notes

3. Consultation

- Initiate video call / see patient in-person
- Discuss symptoms and perform assessment
- Review lab reports and previous history
- Make diagnosis

4. Prescription Creation

- Create digital prescription
- Add medicines with dosage instructions
- Recommend lab tests if required

- Add consultation notes
- Generate PDF prescription

5. Post-Consultation

- Update appointment status to "Completed"
- Set follow-up date if required
- Respond to patient follow-up questions (chat)

6. End of Day

- Review daily revenue dashboard
 - Check patient satisfaction ratings
 - Plan next day's schedule
-

12. TECHNOLOGY STACK

12.1 Backend

- **Framework:** Spring Boot 3.x (Java 17)
- **Security:** Spring Security with JWT
- **Database:** MySQL 8.0
- **Caching:** Redis 7.x
- **File Storage:** AWS S3 / MinIO
- **API Documentation:** Swagger/OpenAPI 3.0

12.2 Frontend

- **Framework:** Angular 17+
- **UI Library:** Angular Material / PrimeNG
- **State Management:** NgRx
- **HTTP Client:** Angular HttpClient with Interceptors
- **Styling:** Tailwind CSS / SCSS

12.3 Mobile App

- **Approach:** Progressive Web App (PWA) with Angular
- **Alternative:** React Native (future consideration)
- **Push Notifications:** Firebase Cloud Messaging (FCM)

12.4 DevOps & Infrastructure

- **Cloud Provider:** AWS / Google Cloud Platform

- **Containerization:** Docker
- **Orchestration:** Kubernetes (future) / Docker Compose (initial)
- **CI/CD:** GitHub Actions / Jenkins
- **Monitoring:** Prometheus + Grafana / AWS CloudWatch
- **Logging:** ELK Stack (Elasticsearch, Logstash, Kibana)

12.5 Third-Party Integrations

- **Payment Gateway:** Razorpay / Paytm
- **Video Calling:** Twilio Video / Agora.io
- **SMS Gateway:** Twilio / AWS SNS
- **Email Service:** SendGrid / AWS SES
- **Maps:** Google Maps API (doctor location)
- **Analytics:** Google Analytics / Mixpanel

12.6 Development Tools

- **IDE:** IntelliJ IDEA / VS Code
 - **API Testing:** Postman / Insomnia
 - **Version Control:** Git / GitHub
 - **Project Management:** Jira / Linear
 - **Design:** Figma
-

13. DEVELOPMENT ROADMAP

Phase 1: Foundation (Months 1-2)

Week 1-2: Setup & Planning

- Finalize technical architecture
- Set up development environment
- Create Git repositories
- Database schema design
- Set up CI/CD pipelines

Week 3-4: Core User Management

- User registration and authentication APIs
- Patient, Doctor, Admin profile APIs

- JWT implementation
- Email/SMS OTP verification
- Basic frontend login/signup pages

Week 5-6: Doctor Discovery & Profiles

- Doctor listing with filters
- Doctor profile pages
- Search functionality
- Doctor availability management
- Frontend doctor discovery UI

Week 7-8: Appointment Booking

- Appointment booking APIs
- Slot management system
- Appointment listing for patients and doctors
- Frontend booking flow
- Calendar integration

Phase 2: Core Features (Months 3-4)

Week 9-10: Payment Integration

- Razorpay/Paytm integration
- Payment initiation and callback APIs
- Invoice generation
- Payment history
- Refund processing
- Frontend payment flow

Week 11-12: Medical Records

- File upload system (S3/MinIO integration)
- Medical records CRUD APIs
- Record viewing and downloading
- Frontend medical records management

Week 13-14: Prescription System

- Prescription creation APIs

- Medicine and lab test management
- PDF generation with digital signature
- Frontend prescription forms
- Prescription viewing for patients

Week 15-16: Notifications

- Push notification setup (FCM)
- SMS integration (Twilio)
- Email notifications (SendGrid)
- Notification APIs
- Appointment reminders automation
- Frontend notification center

Phase 3: Advanced Features (Month 5)

Week 17-18: Teleconsultation

- Video calling integration (Twilio/Agora)
- Call session management
- In-call chat
- Call recording (with consent)
- Frontend video call interface

Week 19-20: Reviews & Ratings

- Review submission APIs
- Rating system for doctors
- Review moderation (admin)
- Frontend review forms
- Doctor rating display

Phase 4: Analytics & Admin (Month 6)

Week 21-22: Doctor Dashboard

- ♦ Revenue analytics
- ♦ Appointment statistics
- ♦ Patient demographics
- ♦ Performance metrics

- Frontend dashboard with charts (Recharts)

Week 23-24: Admin Panel

- User management interface
- Doctor verification workflow
- Platform analytics dashboard
- System monitoring tools
- Content management

Phase 5: Testing & Launch (Month 7)

Week 25-26: Testing

- Unit testing (JUnit, Mockito)
- Integration testing
- Frontend testing (Jasmine, Karma)
- End-to-end testing (Selenium/Cypress)
- Performance testing (JMeter)
- Security testing

Week 27: Beta Launch

- Deploy to staging environment
- Onboard 50 beta users (25 patients, 25 doctors)
- Collect feedback and iterate
- Bug fixes and performance optimization

Week 28: Production Launch

- Deploy to production
- Marketing and user acquisition
- 24/7 monitoring
- Support team ready

Post-Launch Roadmap (Months 8-12)

Month 8-9: Enhancements

- Multi-language support (Hindi, regional languages)
- Lab test booking integration
- Medicine delivery integration

- Health insurance integration
- Family member profiles

Month 10-11: Expansion

- NDHM (National Digital Health Mission) integration
- ABDM (Ayushman Bharat Digital Mission) compliance
- Pharmacy partnerships
- Diagnostic center partnerships
- Health camps and screening programs

Month 12: Advanced Features

- AI-based symptom checker
- Health tracking and monitoring
- Wearable device integration
- Emergency SOS feature
- Second opinion feature

14. TEST STRATEGY

14.1 Test Types

Unit Testing

Coverage Target: 80% code coverage

Backend (JUnit 5 + Mockito):

```
java
```

```

@SpringBootTest
class AppointmentServiceTest
{
    @Mock
    private AppointmentRepository appointmentRepository;

    @InjectMocks
    private AppointmentService appointmentService;

    @Test
    void testBookAppointment_Success() {
        // Test implementation
    }

    @Test
    void testBookAppointment_SlotNotAvailable() {
        // Test implementation
    }
}

```

Frontend (Jasmine + Karma):

```

typescript

describe('AppointmentBookingComponent', () =>
{
    it('should create appointment successfully', () =>
    {
        // Test implementation
    });
});

```

Integration Testing

- API endpoint testing with MockMvc
- Database integration tests
- External service integration tests (payment, SMS, email)
- Frontend-backend integration tests

Functional Testing

Sample Test Cases:

Test ID	Module	Test Case	Priority
TC001	Authentication	User registration with valid details	High
TC002	Authentication	Login with invalid credentials	High
TC003	Appointment	Book appointment with available slot	High
TC004	Appointment	Book appointment with unavailable slot (should fail)	High
TC005	Payment	Process payment successfully	Critical
TC006	Payment	Handle payment failure gracefully	Critical
TC007	Prescription	Create prescription with medicines	High
TC008	Medical Records	Upload and view medical record	High
TC009	Teleconsultation	Initiate video call successfully	High
TC010	Notifications	Send appointment reminder	Medium
TC011	Search	Search doctors by specialization	Medium
TC012	Reviews	Submit review after appointment	Low

Non-Functional Testing

Performance Testing (JMeter):

- Load Testing: 10,000 concurrent users
- Stress Testing: Peak load simulation
- Endurance Testing: 24-hour sustained load
- API response time benchmarks

Security Testing:

- OWASP Top 10 vulnerability assessment
- SQL injection testing
- XSS attack prevention testing
- Authentication and authorization testing
- Data encryption verification
- Penetration testing

Usability Testing:

- User flow testing with real users
- Accessibility testing (screen readers, keyboard navigation)
- Cross-browser testing (Chrome, Safari, Firefox, Edge)
- Mobile responsiveness testing
- Low bandwidth testing

Compatibility Testing:

- Android 10+ devices
- iOS 14+ devices
- Desktop browsers (latest 2 versions)
- Tablet devices

14.2 User Acceptance Testing (UAT)

Phase 1: Internal UAT (Week 25)

- Internal team testing
- Stakeholder review
- Bug fixing

Phase 2: Beta UAT (Week 26-27)

- 25 patients (different age groups, locations)
- 25 doctors (different specializations)
- Real-world scenario testing
- Feedback collection via surveys
- Iterative improvements

UAT Success Criteria:

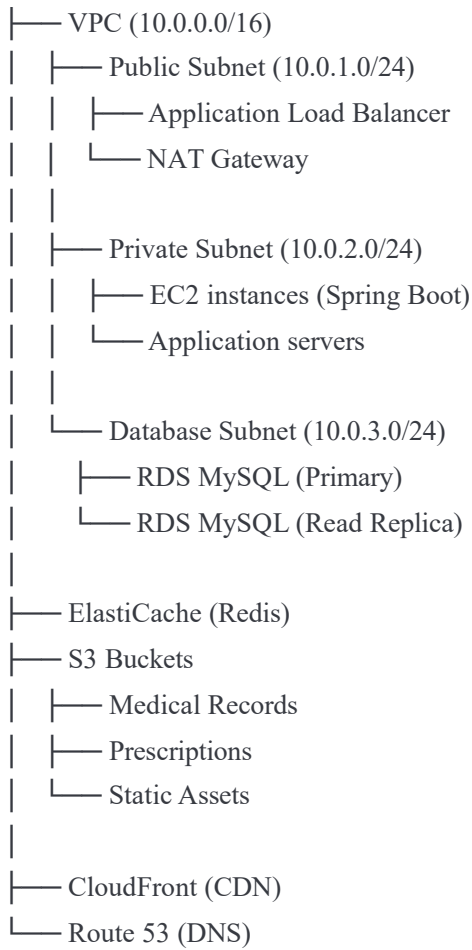
- ♦ 90% task completion rate
 - ♦ User satisfaction score $\geq 4.0/5.0$
 - ♦ < 5 critical bugs
 - ♦ < 20 medium priority bugs
-

15. DEPLOYMENT PLAN

15.1 Infrastructure Setup

AWS Architecture:

Region: ap-south-1 (Mumbai)



Server Specifications:

Environment	Type	Count	Specs
Production API	t3.large	3	2 vCPU, 8GB RAM
Production DB	db.t3.large	1 primary + 1 replica	2 vCPU, 8GB RAM, 100GB SSD
Redis Cache	cache.t3.medium	1	2 vCPU, 3.2GB RAM
Staging	t3.medium	2	2 vCPU, 4GB RAM

15.2 Deployment Process

CI/CD Pipeline (GitHub Actions):

yaml

```
# .github/workflows/deploy.yml
```

```
name: Deploy to Production
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
jobs:
```

```
  build-and-deploy:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

- name: Checkout code
- name: Run tests
- name: Build Docker image
- name: Push to ECR
- name: Deploy to EC2
- name: Health check
- name: Rollback on failure

Deployment Steps:

1. Code merged to main branch
2. Automated tests run
3. Docker image built and pushed to registry
4. Blue-green deployment to EC2 instances
5. Health checks performed
6. Traffic switched to new version
7. Old version kept for 24 hours (rollback capability)

15.3 Monitoring & Alerts

Monitoring Tools:

- Application: AWS CloudWatch / Prometheus
- Database: RDS Performance Insights
- Errors: Sentry
- Uptime: Pingdom / UptimeRobot
- User Analytics: Google Analytics + Mixpanel

Alert Configuration:

Alert Type	Threshold	Action
API Error Rate	> 5%	Slack alert + Email to on-call engineer
Response Time	> 3 seconds	Slack alert
CPU Usage	> 80%	Auto-scaling + Alert
Memory Usage	> 85%	Alert to ops team
Database Connections	> 90%	Critical alert
Payment Failures	> 10%	Immediate escalation
Server Down	Any instance	PagerDuty alert

15.4 Backup & Disaster Recovery

Backup Strategy:

- Database: Automated daily backups at 2 AM IST, retained for 30 days
- Point-in-time recovery enabled (up to 5 minutes)
- Medical records: S3 with versioning and cross-region replication
- Application code: Git repository with tags for each release

Disaster Recovery Plan:

- **RTO (Recovery Time Objective):** 4 hours
- **RPO (Recovery Point Objective):** 1 hour
- Automated failover to read replica (promoted to primary)
- Cross-region backup restoration procedure
- Documented runbook for disaster scenarios

15.5 Post-Deployment Checklist

- ☐ All services healthy and responding
- ☐ Database connections stable
- ☐ SSL certificates valid
- ☐ DNS propagation complete
- ☐ CDN cache purged
- ☐ Monitoring dashboards configured
- ☐ Alert rules active
- ☐ Backup jobs scheduled
- ☐ Security groups configured
- ☐ API rate limits applied
- ☐ Payment gateway in live mode
- ☐ Support team trained and ready
- ☐ Documentation updated

16. RISK ASSESSMENT

16.1 Technical Risks

Risk	Probability	Impact	Mitigation Strategy
Database performance degradation	Medium	High	Implement caching, database indexing, read replicas
Third-party API failures (Payment/Video)	Medium	Critical	Circuit breaker pattern, fallback mechanisms, multiple provider options
Security breach / Data leak	Low	Critical	Regular security audits, penetration testing, encryption, access controls
Server downtime	Low	High	Load balancing, auto-scaling, multi-AZ deployment
Scalability issues during peak load	Medium	High	Performance testing, horizontal scaling, CDN for static content
Video call quality issues	High	Medium	Multiple video providers, bandwidth detection, fallback to audio
Integration delays with NDHM	Medium	Medium	Modular design, API abstraction layer

16.2 Business Risks

Risk	Probability	Impact	Mitigation Strategy
Low doctor adoption	Medium	Critical	Incentive programs, easy onboarding, dedicated support
Low patient trust in teleconsultation	High	High	Doctor verification badges, user reviews, free trial consultations
Regulatory compliance issues	Low	Critical	Legal consultation, DPDp compliance, regular audits
Competition from established players	High	Medium	Unique value proposition (Tier 2/3 focus), competitive pricing
Payment gateway disputes	Medium	Medium	Clear refund policy, escrow mechanism, dispute resolution process
Medical malpractice concerns	Low	Critical	Doctor verification, consultation recordings (with consent), insurance

16.3 Operational Risks

Risk	Probability	Impact	Mitigation Strategy
Inadequate customer support	Medium	High	24/7 support team, comprehensive FAQ, chatbot for common queries

Risk	Probability	Impact	Mitigation Strategy
Doctor credential verification delays	Medium	Medium	Automated verification APIs, partnerships with medical councils
Poor user experience leading to churn	High	High	User testing, feedback loops, continuous UX improvements
Network issues in rural areas	High	Medium	Low bandwidth mode, offline record viewing, SMS notifications

17. BUDGET & RESOURCES

17.1 Team Structure

Development Team (7 months):

Role	Count	Responsibility
Backend Developer (Spring Boot)	2	API development, database design, integrations
Frontend Developer (Angular)	2	Web and mobile app development, UI/UX implementation
UI/UX Designer	1	Design system, wireframes, prototypes
DevOps Engineer	1	Infrastructure, CI/CD, monitoring
QA Engineer	1	Testing, automation, quality assurance
Project Manager	1	Planning, coordination, stakeholder management
Total	8	

Post-Launch Team:

Role	Count
Backend Developer	1
Frontend Developer	1
Customer Support	2
Operations Manager	1

17.2 Infrastructure Cost (Monthly)

AWS Services (Estimated):

Service	Specifications	Monthly Cost (USD)
EC2 (Application)	3 × t3.large	\$250
RDS MySQL	db.t3.large + replica	\$300
ElastiCache Redis	cache.t3.medium	\$100
S3 Storage	1TB + data transfer	\$50
CloudFront CDN	500GB/month	\$50
Load Balancer	1 ALB	\$30
Route 53	DNS hosting	\$5
CloudWatch	Monitoring & logs	\$50
Backup & Snapshots	Automated backups	\$40
Total AWS		\$875

Third-Party Services:

Service	Monthly Cost (USD)
Razorpay (Payment)	2% per transaction
Twilio (Video)	\$500 (estimated for 1000 hours)
SendGrid (Email)	\$50
SMS Gateway	\$100
Sentry (Error Tracking)	\$50
Total Services	~\$700

Total Monthly Infra Cost: ~\$1,575

17.3 Development Cost (7 months)

Assumptions:

- Average developer salary: \$3,000/month (India)
- PM/Designer salary: \$2,500/month
- DevOps/QA: \$2,500/month

Phase	Duration	Team Cost
Development (Months 1-6)	6 months	\$144,000
Testing & Launch (Month 7)	1 month	\$24,000
Infrastructure (7 months)	7 months	\$11,025
Total Development Cost		\$179,025

17.4 First Year Budget

Category	Cost (USD)
Development (7 months)	\$179,025
Operations (5 months post-launch)	\$60,000
Marketing & User Acquisition	\$30,000
Legal & Compliance	\$10,000
Contingency (15%)	\$41,854
Total First Year	\$320,879

Revenue Projections (First Year):

- Platform commission: 15% per consultation
- Average consultation fee: ₹800 (\$10)
- Target: 10,000 appointments/month by Month 12
- Monthly revenue: \$15,000
- Annual revenue: ~\$75,000 (ramping up)

Break-even: Expected in Year 2

18. SUCCESS METRICS & KPIs

18.1 User Acquisition Metrics

Metric	Month 3 Target	Month 6 Target	Month 12 Target
Total Registered Patients	5,000	25,000	100,000
Total Onboarded Doctors	50	250	1,000
Daily Active Users (DAU)	200	1,500	8,000
Monthly Active Users (MAU)	1,500	10,000	50,000

18.2 Engagement Metrics

Metric	Target
Average Sessions per User (Monthly)	3+
Average Session Duration	8+ minutes
Appointment Booking Conversion Rate	25%
Repeat Appointment Rate	40%
Doctor Profile View to Booking Ratio	1:4

18.3 Business Metrics

Metric	Month 3	Month 6	Month 12
Monthly Appointments	500	5,000	10,000
Teleconsultation %	60%	70%	75%
Average Revenue per Appointment	\$1.5	\$1.5	\$1.5
Monthly Recurring Revenue (MRR)	\$750	\$7,500	\$15,000
Customer Lifetime Value (LTV)	\$20	\$25	\$30

18.4 Quality Metrics

Metric	Target
Platform Uptime	99.5%+
API Response Time (P95)	< 2 seconds
Payment Success Rate	98%+
User Satisfaction (CSAT)	4.2+/5.0
Net Promoter Score (NPS)	50+
Doctor Satisfaction Score	4.0+/5.0
Appointment Cancellation Rate	< 10%
Video Call Success Rate	95%+

18.5 Support Metrics

Metric	Target
Average Response Time	< 2 hours
First Contact Resolution	70%+
Support Ticket Volume	< 5% of active users
Average Resolution Time	< 24 hours

19. APPENDIX

19.1 Glossary

- **ABDM:** Ayushman Bharat Digital Mission - India's national digital health ecosystem
- **API:** Application Programming Interface
- **CDN:** Content Delivery Network
- **DPDP:** Digital Personal Data Protection Act
- **JWT:** JSON Web Token - Authentication mechanism

- **NDHM:** National Digital Health Mission
- **OTP:** One-Time Password
- **PCI DSS:** Payment Card Industry Data Security Standard
- **RPO:** Recovery Point Objective
- **RTO:** Recovery Time Objective
- **UAT:** User Acceptance Testing
- **WCAG:** Web Content Accessibility Guidelines

19.2 References

- Spring Boot Documentation: <https://spring.io/projects/spring-boot>
- Angular Documentation: <https://angular.io/docs>
- MySQL Documentation: <https://dev.mysql.com/doc/>
- Razorpay API Docs: <https://razorpay.com/docs/>
- Twilio Video API: <https://www.twilio.com/docs/video>
- ABDM APIs: <https://abdm.gov.in/>

19.3 Contact Information

Project Team:

- Project Manager: Prashant Nagar
- Technical Lead: Gaurav Arora
- Email: project-team@smarthealthcare.in
- Support: support@smarthealthcare.in

19.4 Document Approval

Role	Name	Signature	Date
Project Sponsor			
Project Manager	Prashant Nagar		Oct 01, 2025

20. NEXT STEPS

Immediate Actions (Week 1-2)

1. Stakeholder Review:

- Present document to stakeholders

- Gather feedback and incorporate changes
- Get formal approval

2. Team Formation:

- Hire development team
- Set up communication channels (Slack, Jira)
- Kickoff meeting

3. Technical Setup:

- Provision AWS accounts
- Set up Git repositories
- Configure development environments
- Create database schemas

4. Design Phase:

- Finalize UI/UX designs in Figma
- Create design system and component library
- Get user feedback on mockups

5. Legal & Compliance:

- Consult healthcare law expert
- Draft Terms of Service and Privacy Policy
- Plan DPDP compliance implementation

Key Decisions Required

- ☐ Finalize cloud provider (AWS vs GCP)
 - ☐ Choose video calling provider (Twilio vs Agora)
 - ☐ Select payment gateway (Razorpay vs Paytm)
 - ☐ Decide on mobile app approach (PWA vs Native)
 - ☐ Confirm budget allocation
 - ☐ Approve development timeline
 - ☐ Define MVP feature set if timeline is aggressive
-

CONCLUSION

Smart Healthcare platform addresses a critical gap in India's healthcare ecosystem by connecting patients in Tier 2/3 cities with specialist doctors from metros. With a comprehensive feature set covering appointment booking, teleconsultation, digital health records, and transparent billing, the platform is positioned to transform healthcare access for millions.

The technical architecture leverages proven technologies (Spring Boot, Angular, MySQL) ensuring stability and scalability. The 7-month development roadmap with phased delivery minimizes risk while delivering value incrementally.

With a projected break-even in Year 2 and potential to serve 100K+ patients annually, Smart Healthcare represents both a socially impactful initiative and a viable business opportunity.

This document is ready for stakeholder presentation, investor pitch, and development team execution.

Document Version: 1.0

Last Updated: October 01, 2025

Status: Ready for Review & Approval